

## EXPERIMENT 4

**Aim:** Study of solidity program structure, basic constructs compilation and deployment environment.

### Theory :

Solidity is a statically typed, contract-oriented programming language for writing smart contracts on Ethereum and other EVM-compatible blockchains. A smart contract is a program stored on the blockchain that runs exactly as written, without a trusted third party. Remix IDE is a browser-based development environment for writing, compiling, deploying and testing Solidity code. It includes the **Remix VM**, a simulated in-browser blockchain with test accounts that hold 100 test Ether each. A Solidity file starts with a license identifier and a pragma version statement, followed by one or more contract definitions:

### Primitive Data Types, Variables, Functions (pure, view)

- **Data types:** uint (unsigned integer, alias for uint256), int (signed integer), bool, address (20-byte account address), bytes and string. Unassigned variables take default values (0, false, zero address).
- **Variables:**
  - *State variables* are declared outside functions and stored permanently on the blockchain.
  - *Local variables* are declared inside functions and are not saved.
  - *Global variables* give blockchain information, such as msg.sender, msg.value, block.timestamp and block.number.
- **Functions:** A function that changes state needs a transaction and costs gas. A view function only reads state, and a pure function neither reads nor modifies state. Both are free when called externally.

### Inputs and Outputs to Functions

Functions can take multiple input parameters and return multiple values, including tuples. Return variables can be named, and then no return statement is needed (Tutorial 8: returnTwo). Arrays can be used as inputs and outputs, but large arrays are costly in gas. Mappings cannot be used as parameters or return values of public functions.

### Visibility, Modifiers and Constructors

| Visibility | Accessible from                                       |
|------------|-------------------------------------------------------|
| public     | Inside the contract, derived contracts and externally |
| private    | Only the same contract                                |
| internal   | Same contract and contracts inheriting from it        |

|          |                                                          |
|----------|----------------------------------------------------------|
| external | Only from outside; not callable internally without this. |
|----------|----------------------------------------------------------|

- **Modifiers** are reusable code blocks that run before and/or after a function body, with `_;` marking where the function body executes. They are used for access control (`onlyOwner`), input validation (`require`) and guarding against reentrancy.
- **Constructors** run only once, at deployment, and typically set the owner to `msg.sender`.
- **Inheritance** (contract `Child` is `Base`) lets a child contract reuse the functions and non-private variables of a base contract.

### Control Flow: if-else and Loops

- **Conditionals:** `if`, `else if`, `else`, and the ternary operator `condition ? a : b`.
- **Loops:** `for`, `while` and `do-while`, along with `break` (exit the loop) and `continue` (skip to the next iteration).
- Loops with many iterations or unbounded length consume large amounts of gas and can make a transaction fail, so they should be used with care.

### Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays:** fixed-size (`uint[3]`) or dynamic (`uint[]`). They support `push()`, `pop()`, `length` and `delete`.
- **Mappings:** `mapping(KeyType => ValueType)` hash tables with fast lookup. They cannot be iterated and have no length, and every key returns the default value until set. Nested mappings are allowed.
- **Structs:** custom types that group related variables, such as `struct Todo { string text; bool completed; }`.
- **Enums:** user-defined types with named constants, such as `enum Size { S, M, L }`. They are stored as `uint8`, and the first member is the default.

## 7. Data Locations

| Location | Description                                                              |
|----------|--------------------------------------------------------------------------|
| storage  | Permanent, on the blockchain; used for state variables; highest gas cost |
| memory   | Temporary, exists only during function execution; can be modified        |
| calldata | Read-only, used for external function parameters; cheapest               |

Assigning storage to memory **copies** the data. Assigning a storage reference (`MyStruct storage s = myStructs[1]`) points to the original, so changes affect the blockchain state. Parameters of type storage are allowed only in internal or private functions.

### Transactions: Ether and Wei, Gas and Gas Price, Sending Ether

- **Units:** 1 wei is the smallest unit, 1 gwei =  $10^9$  wei, and 1 ether =  $10^{18}$  wei.
- **Gas:** a measure of computational work. Transaction fee = **gas used** × **gas price**. If a transaction runs out of gas, its state changes are reverted, but the gas spent is not refunded.
- **Sending Ether:** a function must be marked payable to receive Ether. There are three ways to send it:
  - `transfer()` reverts on failure and is limited to 2300 gas.
  - `send()` returns a boolean on failure and is limited to 2300 gas.
  - `call{value: x}("")` forwards available gas, returns a boolean and is the **currently recommended method**. It should be paired with `require(sent, ...)`.

## Code and Output :

The screenshot shows the Remix IDE interface. The top bar includes the Remix logo, version 2.6.5, a search bar, and navigation icons. The left sidebar, titled 'LEARNETH', contains a search bar and a 'Filters' section with checkboxes for 'LEVEL' (Beginner, Intermediate, Advanced) and 'TAGS' (Aave, Applications, Axelar, Balancer, Circom, Code Patterns). The main workspace area is divided into 'START' and 'OPEN' sections. The 'START' section has buttons for 'Create a new workspace', 'Start Coding', and 'Start Learning'. The 'OPEN' section has buttons for 'Open Folder' and 'Clone'. At the bottom, there is a 'DESKTOP APP' section with a checkbox for 'Listen on all transactions' and a search bar for 'Filter with transaction hash or address'.

This screenshot shows the Remix IDE interface with the 'Solidity Beginner Course' selected in the sidebar. The sidebar now displays the course title, a description, and a 'Syllabus' section with a list of 19 chapters: 1. Introduction, 2. Basic Syntax, 3. Primitive Data Types, and 4. Variables. The main workspace area remains the same, with 'START' and 'OPEN' sections for creating or opening workspaces. The 'DESKTOP APP' section at the bottom is also visible.

# CHAPTER 1 :

REMIX 2.6.5 Learneth Solidity Beginner Course AI Code QuickDApp introduction.sol Docgen Viewer

FILE EXPLORER

- Learneth Solidity Beginner Course
  - .deps
  - 1. Introduction
    - introduction.sol
  - artifacts
    - build-info
    - Counter\_metadata.json
    - Counter.json
    - prettierrc.json
    - README.md
    - remix.config.json
    - step.md

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public { infinite gas
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public { infinite gas
19    }
20 }
```

Explain contract

Listen on all transactions Filter with transaction hash or address

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

Scam Alert Initialize as git repo RemixAI Copilot (enabled)

REMIX 2.6.5 Learneth Solidity Beginner Course AI Code QuickDApp introduction.sol Docgen Viewer

DEPLOY & RUN TRANSACTIONS

Environment

Remix VM Osaka

Account 1 99.999 ETH

Deploy Deployed Contracts 1 Transaction History 1

Deployed Contracts 1 + Add Contract

Interact with a deployed contract

Counter 0xd91...39138 2s ago

Functions

- dec Transact
- inc Transact
- count Call

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public { infinite gas
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public { infinite gas
19    }
20 }
```

Explain contract

Listen on all transactions Filter with transaction hash or address

Type the library name to see available commands.

[vm] from: 0x5B3...eddC4 to: Counter.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xc1b...b5b21

Scam Alert Initialize as git repo RemixAI Copilot (enabled)

REMIX 2.6.5 Learneth Solidity Beginner Course

DEPLOY & RUN TRANSACTIONS

Environment: Remix VM, Osaka, Account 1 (0x5B3...eddC4), 99.999 ETH

Deployed Contracts: Counter (0xd91...39138)

Functions: dec, inc, count, get

Transaction History: 38s ago

Code (introduction.sol):

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

execute Javascript scripts:

- Input a script directly in the command line interface
- Select a Javascript file in the file explorer and then run 'remix.execute()' or 'remix.executeCurrent()' in the command line interface
- Right-click on a Javascript file in the file explorer and then click 'Run'

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

[vm] from: 0x5B3...eddC4 to: Counter.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xc1b...b5b21

CALL [call] from: 0x5B380a6a701c568545dcfcb03fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c

[vm] from: 0x5B3...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0x3ec...ee5b5

[vm] from: 0x5B3...eddC4 to: Counter.dec() 0xd91...39138 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0xcc...79d46

Scam Alert Initialize as git repo RemixAI Copilot (enabled)

## CHAPTER 2 :

REMIX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list: 2. Basic Syntax (2 / 19)

We also define the **visibility** of the variable, which specifies from where you can access it. In this case, it's a **public** variable that you can access from inside and outside the contract.

Don't worry if you didn't understand some concepts like **visibility**, **data types**, or **state variables**. We will look into them in the following sections.

To help you understand the code, we will link in all following sections to video tutorials from the [creator](#) of the Solidity by Example contracts.

[Watch a video tutorial on Basic Syntax.](#)

★ Assignment

1. Delete the HelloWorld contract and its content.
2. Create a new contract named "MyContract".
3. The contract should have a public state variable called "name" of the type string.
4. Assign the value "Alice" to your new variable.

Check Answer Show answer

Next

Well done! No errors.

Code (basicSyntax.sol):

```
1 // SPDX-License-Identifier: MIT
2 // compiler version must be greater than or equal to 0.8.3 and less than 0.9.0
3 pragma solidity ^0.8.3;
4
5 contract MyContract {
6     string public name = "Alice";
7 }
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

[vm] from: 0x5B3...eddC4 to: Counter.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xc1b...b5b21

CALL [call] from: 0x5B380a6a701c568545dcfcb03fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c

[vm] from: 0x5B3...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0x3ec...ee5b5

[vm] from: 0x5B3...eddC4 to: Counter.dec() 0xd91...39138 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0xcc...79d46

Scam Alert Initialize as git repo RemixAI Copilot (enabled)

## CHAPTER 3 :

REMUX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

3. Primitive Data Types 3 / 19

All these data types have default values, as shown in the contract (line 29):

You can learn more about these data types as well as *Fixed Point Numbers*, *Byte Arrays*, *Strings*, and more in the [Solidity documentation](#).

Later in the course, we will look at data structures like **Mappings**, **Arrays**, **Enums**, and **Structs**.

Watch a video tutorial on [Primitive Data Types](#).

★ Assignment

1. Create a new variable `newAddr` that is a `public address` and give it a value that is not the same as the available variable `addr`.
2. Create a `public` variable called `neg` that is a negative number, decide upon the type.
3. Create a new variable, `newU` that has the smallest `uint` size type and the smallest `uint` value and is `public`.

Tip: Look at the other address in the contract or search the internet for an Ethereum address.

Check Answer Show answer

Next

Well done! No errors.

Compile Docgen Viewer basicSyntax.sol basicSyntax\_answer.sol primitiveDataTypes.sol

```
24 int public i256 = 456;
25 int public i = -123; // int is same as int256
26
27 address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733c;
28
29 // Default values
30 // Unassigned variables have a default value
31 bool public defaultBool; // false
32 uint public defaultUint; // 0
33 int public defaultInt; // 0
34 address public defaultAddr; // 0x0000000000000000000000000000000000000000
35 address public newAddr = 0x0000000000000000000000000000000000000000;
36 int public neg = -10;
37 uint8 public newU = 0; // smallest uint type and smallest value
38 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo RemixAI Copilot (enabled)

## CHAPTER 4 :

REMUX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

4. Variables 4 / 19

retrieve information about the blockchain, particular addresses, contracts, and transactions.

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ Assignment

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer Show answer

Next

Well done! No errors.

Compile Docgen Viewer basicSyntax.sol basicSyntax\_answer.sol primitiveDataTypes.sol variables.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     string public text = "Hello";
7     uint public num = 123;
8
9     uint public blockNumber;
10    function doSomething() public { 22300 gas
11        blockNumber = block.number;
12    }
13
14 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo RemixAI Copilot (enabled)

## CHAPTER 5 :

REMUX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

5.1 Functions - Reading and Writing to a State Variable 5 / 19

If the function takes inputs like our `set` function (line 9), you must specify the parameter types and names. A common convention is to use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections. Watch a video tutorial on Functions.

★ Assignment

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7     bool public b = true;
8     function get_b() public view returns (bool) { return b; } 2539 gas
9
10    // You need to send a transaction to write to a state variable.
11    function set(uint _num) public { 22536 gas
12        num = _num;
13    }
14
15    // You can read from a state variable without sending a transaction.
16    function get() public view returns (uint) { 2475 gas
17        return num;
18    }
19 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo RemixAI Copilot (enabled)

## CHAPTER 6 :

REMUX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

5.2 Functions - View and Pure 6 / 19

From the Solidity documentation.

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions `view` and `pure` can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments. Watch a video tutorial on View and Pure Functions.

★ Assignment

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addrox(uint y) public view returns (uint) { Infinite gas
9         return x + y;
10    }
11
12    // Promise not to modify or read from the state.
13    function add(uint i, uint j) public pure returns (uint) { Infinite gas
14        return i + j;
15    }
16    function addToX2(uint y) public { Infinite gas
17        x = x + y;
18    }
19 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo RemixAI Copilot (enabled)

## CHAPTER 7:

REMUX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

5.3 Functions - Modifiers and Constructors 7 / 19

contract initialization code. The constructor can have parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

★ Assignment

1. Create a new function, `increaseX`, in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
2. Make sure that `x` can only be increased.
3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer Show answer

Next

Well done! No errors.

Scam Alert Initialize as git repo

modifiersAndConstructors.sol

```
47
48
49     function decrement(uint i) public noReentrancy {
50         x -= i;
51         if (i > 1) {
52             decrement(i - 1);
53         }
54     }
55     modifier greaterthan0(uint y) {
56         require(y > 0, "Input parameter not greater than 0");
57         _;
58     }
59     modifier actualincrease(uint y) {
60         _;
61         x = x + y;
62     }
63
64     function increaseX(uint y) public onlyOwner greaterthan0(y) actualincrease(y) {
65         // body intentionally empty
66     }
67 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

RemixAI Copilot (enabled)

## CHAPTER 8:

REMUX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

5.4 Functions - Inputs and Outputs 8 / 19

functions.

"[Mappings] cannot be used as parameters or return parameters of contract functions that are publicly visible." From the Solidity documentation.

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

Watch a video tutorial on Function Outputs.

★ Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

Check Answer Show answer

Next

Well done! No errors.

Scam Alert Initialize as git repo

inputsAndOutputs.sol

```
68 // cannot use map for neither input nor output
69
70 // Can use array for input
71 function arrayInput(uint[] memory _arr) public {
72     // Can use array for output
73     uint[] public arr;
74
75     function arrayOutput() public view returns (uint[] memory) {
76         return arr;
77     }
78
79     function returnTwo() public pure returns (int p, bool q) {
80         p = -2;
81         q = true;
82     }
83 }
84 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

RemixAI Copilot (enabled)



## CHAPTER 9 :

REMUX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

6. Visibility 9 / 19

State variables can not be `external`

In this example, we have two contracts, the `Base` contract (line 4) and the `Child` contract (line 55) which inherits the functions and state variables from the `Base` contract.

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

Watch a video tutorial on Visibility.

★ Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer Show answer

Next

Well done! No errors.

```
contract Child is Base {
  // Inherited contracts do not have access to private functions
  // and state variables.
  // function testPrivateFunc() public pure returns (string memory) {
  //   return privateFunc();
  // }

  // Internal function call be called inside child contracts.
  function testInternalFunc() public pure override returns (string memory) {
    return internalFunc();
  }

  function testInternalVar() public view returns (string memory, string memory) {
    return (internalVar, publicVar);
  }
}
```

AI copilot

0 Listen on all transactions Filter with transaction hash or address

RemixAI Copilot (enabled)

## CHAPTER 10 :

REMUX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

7.1 Control Flow - If/Else 10 / 19

else if

With the `else if` statement we can combine several conditions.

If the first condition (line 6) of the `foo` function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the If/Else statement.

★ Assignment

Create a new function called `evenCheck` in the `ifElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer Show answer

Next

Well done! No errors.

```
// if (x < 10) {
//   return 1;
// }
// return 2;

// shorthand way to write if / else statement
return _x < 10 ? 1 : 2;

function evenCheck(uint a) public pure returns (bool) {
  return a % 2 == 0 ? true : false;
}
```

AI copilot

0 Listen on all transactions Filter with transaction hash or address

RemixAI Copilot (enabled)

## CHAPTER 11 :

REMIX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

7.2 Control Flow - Loops 11 / 19

The `continue` statement is used to skip the remaining code block and start the next iteration of the loop. In this contract, the `continue` statement (line 10) will prevent the second if statement (line 12) from being executed.

**break**

The `break` statement is used to exit a loop. In this contract, the `break` statement (line 14) will cause the for loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

★ Assignment

1. Create a public `uint` state variable called `count` in the `Loop` contract.
2. At the end of the for loop, increment the count variable by 1.
3. Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer Show answer

Next

Well done! No errors.

```
10  while (i < 10) {
11      if (i == 3) {
12          // Skip to next iteration with continue
13          continue;
14      }
15      if (i == 5) {
16          // Exit loop with break
17          break;
18      }
19      count++;
20  }
21
22  // while loop
23  uint j;
24  while (j < 5) {
25      j++;
26      count++;
27  }
28
29  function getArr() public view returns (uint[3] memory) { Infinite gas
30      return arr3;
31  }
32  }
```

AI Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo RemixAI Copilot (enabled)

## CHAPTER 12 :

REMIX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

8.1 Data Structures - Arrays 12 / 19

same, which means that the length of the array will stay the same. This will create a gap in our array. If the order of the array is not important, then we can move the last element of the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

**Array length**

Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

★ Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer Show answer

Next

Well done! No errors.

```
34  }
35
36  function getLength() public view returns (uint) { 2505 gas
37      return arr.length;
38  }
39
40  function remove(uint index) public { 7631 gas
41      // Delete does not change the array length.
42      // It resets the value at index to it's default value,
43      // in this case 0
44      delete arr[index];
45  }
46
47  function getArr() public view returns (uint[3] memory) { Infinite gas
48      return arr3;
49  }
50
51  contract CompactArray {
52      uint[] public arr;
53
54      // Deleting an element creates a gap in the array.
55      // One trick to keep the array compact is to
56      // move the last element into the place to delete.
57      function remove(uint index) public { Infinite gas
58          // Move the last element into the place to delete
59      }
```

AI Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo RemixAI Copilot (enabled)

## CHAPTER 13 :

The screenshot shows the Remix IDE interface. On the left, the 'LEARNETH' sidebar displays the '8.2 Data Structures - Mappings' tutorial (13 / 19). The main content area shows the 'Removing values' section, explaining the delete operator and providing an assignment with three steps: creating a mapping, using get and remove functions, and creating a new entry. Below the assignment are 'Check Answer' and 'Show answer' buttons, and a green confirmation message: 'Well done! No errors.' The right pane shows the Solidity code for 'mappings.sol', which defines a 'Mapping' contract with a 'balances' mapping and functions 'get', 'set', and 'remove'. It also includes a 'NestedMapping' contract. The bottom status bar indicates 'RemixAI Copilot (enabled)'.

## CHAPTER 14 :

The screenshot shows the Remix IDE interface. On the left, the 'LEARNETH' sidebar displays the '8.3 Data Structures - Structs' tutorial (14 / 19). The main content area shows the 'Accessing structs' and 'Updating structs' sections, explaining the dot operator and providing an assignment with one step: creating a 'remove' function. Below the assignment are 'Check Answer' and 'Show answer' buttons, and a green confirmation message: 'Well done! No errors.' The right pane shows the Solidity code for 'structs.sol', which defines a 'struct' and functions 'update', 'toggleCompleted', and 'remove'. The bottom status bar indicates 'RemixAI Copilot (enabled)'.

## CHAPTER 15 :

REMITX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

8.4 Data Structures - Enums 15 / 19

We can update the enum value of a variable by assigning it the `uint` representing the enum member (line 30). Shipped would be 1 in this example. Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

**Removing an enum value**

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

Watch a video tutorial on Enums.

★ Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer Show answer

Next

Well done! No errors.

Scam Alert Initialize as git repo

```
4 contract Enum {
5   // Enum representing shipping status
6   enum Status {
7     Pending,
8     Shipped,
9     Accepted,
10    Rejected,
11    Canceled
12  }
13   enum Size { S, M, L }
14   Size public sizes;
15
16   function getSize() public view returns (Size) {
17     return sizes;
18   }
19
20   // Default value is the first element listed in
21   // definition of the type, in this case "Pending"
22   Status public status;
23
24   // Returns uint
25   // Pending - 0
26   // Shipped - 1
27   // Accepted - 2
28   // Rejected - 3
29 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

RemixAI Copilot (enabled)

## CHAPTER 16 :

REMITX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

9. Data Locations 16 / 19

As we said in the beginning, when creating contracts we have to be mindful of gas costs. Therefore, we need to use data locations that require the lowest amount of gas possible.

★ Assignment

1. Change the value of the `myStruct` member `foo` inside the `function f` to 4.
2. Create a new struct `myMemStruct2` with the data location `memory` inside the `function f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer Show answer

Next

Well done! No errors.

Scam Alert Initialize as git repo

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract DataLocations {
5   uint[] public arr;
6   mapping(uint => address) map;
7   struct MyStruct {
8     uint foo;
9   }
10   mapping(uint => MyStruct) public myStructs;
11
12   function f() public returns (MyStruct memory, MyStruct memory, MyStruct memory) {
13     // call _f with state variables
14     _f(arr, map, myStructs[1]);
15
16     // get a struct from a mapping
17     MyStruct storage myStruct = myStructs[1];
18     myStruct.foo = 4;
19
20     // create a struct in memory
21     MyStruct memory myMemStruct = MyStruct(0);
22     MyStruct memory myMemStruct2 = myMemStruct;
23     myMemStruct2.foo = 1;
24 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

RemixAI Copilot (enabled)

## CHAPTER 17 :

REMITX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

10.1 Transactions - Ether and Wei 17 / 19

without a suffix are treated as `wei` (line 7).

One `wei` (giga-wei) is equal to 1,000,000,000 ( $10^9$ ) `wei`.

One `ether` is equal to 1,000,000,000,000,000,000 ( $10^{18}$ ) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

★ Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and  $10^9$ .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12    uint public oneGwei = 1 gwei;
13    bool public isOneGwei = 1 gwei == 1e9;
14 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo

RemixAI Copilot (enabled)

## CHAPTER 18 :

REMITX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

10.2 Transactions - Gas and Gas Price 18 / 19

When sending a transaction, the sender specifies the maximum amount of gas that they are willing to pay for. If they set the limit too low, their transaction can run out of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.

Learn more about gas on [ethereum.org](https://ethereum.org).

Watch a video tutorial on Gas and Gas Price.

★ Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public {
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        while (true) {
15            i += 1;
16        }
17    }
18 }
19
```

Explain contract

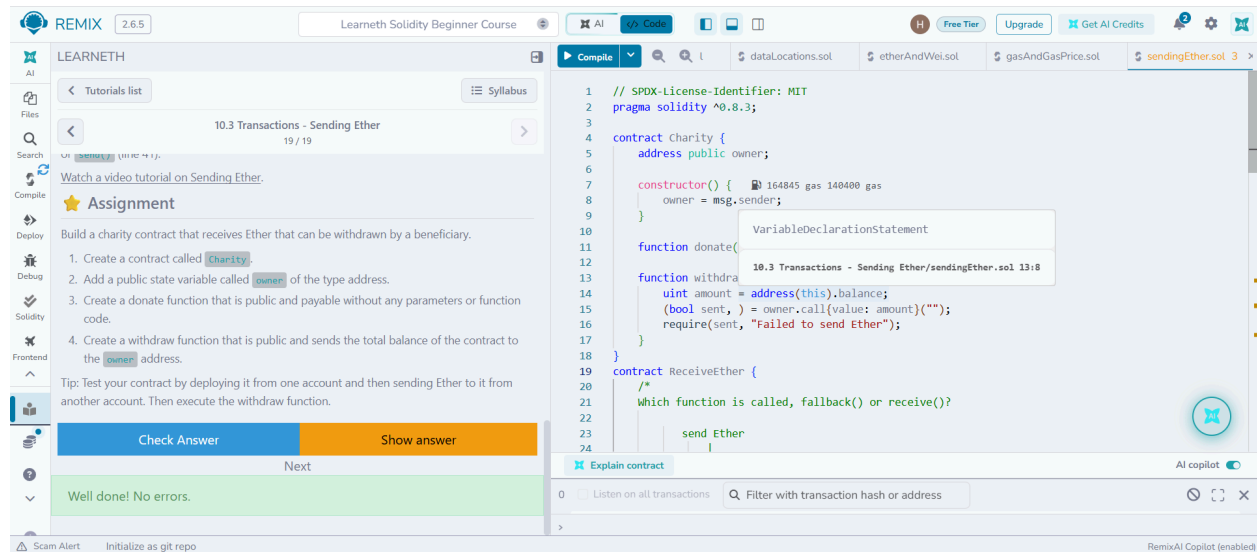
AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo

RemixAI Copilot (enabled)

## CHAPTER 19 :



## Conclusion :

In this experiment, the fundamentals of Solidity programming were studied and practiced by completing all 19 assignments of the LearnEthereum *Solidity Basics* course in the Remix IDE. The structure of a Solidity program, including the license identifier, pragma directive and contract definition, was understood, and a Counter contract was compiled, deployed on the Remix VM and tested using get, inc and dec.

The experiment covered primitive data types, state, local and global variables, pure and view functions, multiple inputs and outputs, function visibility (public, private, internal, external), modifiers and constructors for access control, and control flow using conditionals and loops. Data structures (arrays, mappings, structs and enums) were implemented, and the difference between storage, memory and calldata was understood through the DataLocations contract. Finally, Ether and wei units, gas and gas price, and sending Ether using call were applied by building a Charity contract that accepts donations and lets the owner withdraw them.